

INTRODUCTION

In March 2025, attackers modified the tag configuration of `tj-actions/changed-files`, a widely reused GitHub Actions helper, so that 346 existing version — from v39 through v47 — all pointed to a single malicious commit [21]. Any project that were using one of those version ran attacker-controlled code. The malicious code could expose credentials of every affected project.

What is striking about the incident is that it required no software vulnerability. The attack's leverage came entirely from a gap between what Git's reference model permits and what downstream consumers assume. Git tag names can be updated at any time by anyone with push access to a repository. The attacker used that capability to retarget familiar version labels to a commit of their choice. Downstream consumers, for their part, had treated those labels as stable, or at least trustworthy, identifiers — not as mutable pointers whose targets could be silently replaced overnight. If tag names are a social contract about stability, Git does not enforce that contract.

This thesis asks how far that assumption is violated in practice, across the free and open-source software ecosystem, and what the consequences are. Three empirical studies follow the assumption's failure progressively inward: from version tags — the release labels that developers, package managers, and build systems use to pin upstream dependencies — to the underlying commit histories that those labels anchor, and finally to the files that contained credentials and that some maintainers remove from history when they discover they were accidentally committed. Together, the three studies trace a single thread from its most visible surface to its deepest security implication.

n'oublie pas la surete

p-e dire que c'est chercher une aiguille dans une botte de foin
ce n'est pas parce qu'on ne le trouve pas que le probleme ne
peut pas se poser.

1.1 Software Repositories as Load-Bearing Infrastructure

Open-source software now forms the connective tissue of modern computing. Web services, embedded systems, scientific computing environments, and public digital infrastructure all depend, to varying degrees, on software libraries, tools, and operating environments whose source code is publicly maintained in repositories. The scope of this dependency is structural, not merely quantitative. A typical software product today is assembled from hundreds or thousands of upstream packages, each of which may itself depend on further upstream components. The production of a deployable artifact — a compiled binary, a container image, a published package — is therefore a collaborative act that involves contributors and maintainers who have never met, coordinated through the shared convention of version-controlled repositories.

This dependency structure creates a demand for stability. Package managers such as npm, PyPI, crates.io, and Maven Central resolve version specifications against repository state on behalf of millions of developers every day. Continuous integration pipelines re-execute builds against declared dependency sets, expecting identical inputs to produce repeatable outputs. Lockfiles and software bills of materials record the specific sources retrieved during a build so that the same build can be reproduced or audited later. Supply-chain security frameworks such as SLSA (Supply-chain Levels for Software Artifacts) build provenance metadata around the assumption that named source inputs refer to well-defined, stable content [57]. All of these mechanisms share a common structural premise: that resolving a dependency specification today will yield the same artifact as resolving it yesterday.

The stability assumption is reasonable at the level of content-addressed identifiers. If a build system records a cryptographic hash of each upstream source, verifying reproducibility reduces to checking that the same hash is still reachable. But in practice, most dependency specifications and workflow configurations do not pin content hashes. They pin version labels: a package version string such as v2.3.1 or a branch name such as main. These labels are human-readable names, not cryptographic commitments. Their stability is a social convention, not a technical guarantee enforced by the version control system. The ecosystem treats them as durable identifiers because most maintainers honour the convention most of the time — not because Git prevents a name from being moved.

tu explicites la version human :)
mais pas la version crypto :)

This social contract turns public repositories into trust anchors for the software supply chain. When a developer writes tj-actions/changed-files@v45 in a CI workflow, they trust that the upstream repository will not silently retarget that label. When a reproducible-build system records that an artifact was built from a specific tagged commit, it trusts that the tag will not be moved. When a security team audits the dependency tree of a deployed application, it trusts that version numbers recorded at build time correspond to the source code that was actually compiled [61]. None of these trust assumptions is enforced at the protocol level. They are maintained by norms, by the perceived cost of violating them, and by the expectation that repository state is stable enough in practice to be treated as if it were stable by design.

phrase obscure qui « conclut » le paragraphe precedent obscure aussi

This thesis is motivated by the question of whether that expectation is well-calibrated. The tj-actions incident demonstrates that an adversary can exploit the gap between assumption and reality to achieve supply-chain compromise at scale. But the attack is only the most visible manifestation of a broader phenomenon. Repository state changes in practice far beyond headline incidents. Release tags are moved to correct erroneous releases or to automate version management. Commit histories are rewritten to remove debug artifacts, clean up merge noise, or erase accidentally committed credentials. And the files erased during those rewrites sometimes contain sensitive material that remains recoverable from independent archives long after the maintainer believed it had been removed. The security implications of these ordinary practices are poorly understood, not because they are rare, but because the evidence they leave behind disappears from the hosting platform — the *forge*, e.g. GitHub or GitLab — as soon as the repository is updated. Measuring that evidence requires a data source that differs fundamentally from the forge itself.

mal dit : dépasse ?

1.2 The Gap Between Git's Design and Its Consumption

Mais de quoi vas-tu parler ??? Difficile à suivre !

Git stores repository data in a content-addressable object store: every object — file contents, directory trees, commits — is identified by the cryptographic digest of its contents and meta-data [11]. Because a commit's identifier is computed over its parent commit(s) — the commit(s) immediately before it in the project's history — it encodes the entire history reachable from it, and any change to an earlier commit propagates new identifiers to every later commit built on top of it, through what this thesis calls the *snowball effect*. Content cannot be altered without changing its identity: an observer who stores a commit hash and later retrieves the same hash from any clone is guaranteed to obtain the same content.

References — branches, tags, and the symbolic pointers that developers and downstream systems actually use to navigate repository state — do not share this property. A reference is a named pointer to an object identifier: the name is stable, but the object it designates is not. Tags — the labels developers use to mark releases — enjoy no protocol-level protection against retargeting or deletion [11, 22]. The *Pro Git* book itself describes tags as “very much like a branch that doesn't change” [11] — a characterisation that is pedagogically intuitive but technically inaccurate, and that precisely captures the stability assumption that downstream consumers inherit. Chapter 2 develops this object-reference distinction in full.

This gap between the immutability of objects and the mutability of references manifests in three practically distinct dimensions, each studied by one chapter of this thesis. Release tags can be silently retargeted or deleted while the familiar version string remains visible in registries, documentation, and workflow configurations. Commit histories can be rewritten after publication, so that the original commits are no longer reachable from the branch head — the commit a branch name currently points to — and the altered state appears as if it had always been the

CHAPTER 1. INTRODUCTION

one that existed. And history rewriting is itself used as a remediation strategy for accidentally committed credentials — a practice that creates a false sense of security, because any independent system that observed the repository before the rewrite may still hold the original commit and its contents.

These three dimensions share a structural property: they are not exclusively security problems. Tag alterations are often benign corrections. History rewrites are often routine maintenance. Credential file removals are often genuine remediation attempts, carried out in good faith by maintainers who believe that rewriting history is enough on its own, whether or not they also rotate the credential. From an external, observational stance, it is not possible to determine intent from the alteration itself. What all alterations share, regardless of intent, is that they break the stability assumption on which downstream consumers depend, and that the evidence of the break is gone from the forge the moment the repository changes again. Measuring the prevalence, structure, and security implications of these alterations requires a different kind of empirical infrastructure.

1.3 Software Heritage as an Empirical Lever

Software Heritage is the largest public archive of source code [3]. It continuously crawls public code forges — including GitHub, GitLab, Bitbucket, Codeberg, SourceHut, and hundreds of smaller instances — and stores complete snapshots of each repository’s state at the time of each visit. Unlike a forge, which provides access to only the current state of a repository, Software Heritage retains all states it has ever observed. It assigns persistent, content-derived identifiers (SWHIDs) to every archived object, from individual file contents to full repository snapshots, in conformance with an international standard [1]. These identifiers are computed from content and are therefore stable across all copies and all time: the same SWHID will always refer to the same artifact, regardless of when or from where it is retrieved.

The archive’s structure creates a natural instrument for studying repository mutability. Each time Software Heritage visits the same repository more than once, it produces two independent observations of the same origin at different points in time. Comparing consecutive snapshots — a *snapshot pair* — reveals which references existed in both observations, which changed their target, and which appeared or disappeared. Crucially, even if the forge has already overwritten the original state of a tag or branch, the earlier snapshot retains the pre-alteration reference target. The SWHID of the original commit is preserved in the archive, alongside the SWHID of its replacement, making it possible to compare the two states at any later date and to verify that they differ. This before-and-after observability at scale is what makes the measurements reported in this thesis possible.

Prior empirical work on repository mutability was largely confined to single-forge studies, short observation windows, or samples too small to estimate ecosystem-level rates with confidence.

The studies in this thesis operate at a different scale: the tag-alteration study analyses 45.5 M repositories with at least one tag, observed across a nine-year window spanning all major forges; the history-alteration study analyses 111 M repositories with at least two archival visits. These scale differences are not incidental. Mutability that affects a small fraction of repositories in relative terms may still touch millions of real projects in absolute terms, and the shape of the distribution — whether alteration is concentrated in a few outliers or spread across the population — can only be characterised reliably at this order of magnitude. The snapshot-pair methodology, implemented on top of Software Heritage and described in full in Section 3.3, is the shared empirical backbone of all three studies.

1.4 Three Studies of Repository Integrity

The three studies that constitute this thesis are motivated by, and structured around, the three dimensions of mutability introduced in Section 1.2. Each study asks a distinct empirical question, employs the snapshot-pair methodology described in Section 1.3, and produces findings that bear on a different facet of the repository integrity problem. The studies are presented in escalating specificity: the first operates at the level of release labels, the second at the level of commit-graph structure, and the third at the level of file content within altered histories.

1.4.1 When Tag Names Outlive Their Targets

The first study is motivated directly by the `tj`-actions incident and asks whether tag alteration is an isolated phenomenon or a regular feature of the public software ecosystem. Before this work, there was little public evidence about how often the convention of tag stability is broken in ordinary maintenance as opposed to headline attacks, and no comprehensive study had characterised tag alterations at ecosystem scale. The central questions are: How prevalent is tag alteration, and does the practice persist over time? When tags do change, what forms do those changes take? And how do alteration patterns relate to a repository’s downstream visibility?

To answer these questions, the study compares consecutive Software Heritage snapshot pairs for all origins that had been visited at least twice and carried at least one Git tag as of October 8, 2025, yielding a corpus of 45.5 M repositories observed over 2016–2025 [52]. Each tag reference is tracked across successive snapshots: a tag that points to a different commit in two consecutive observations is classified as a *move*; a tag that was present in one snapshot but absent from the next is classified as a *deletion*. Moves are further distinguished according to what changed — whether the source tree changed (a content-changing move), whether only commit metadata such as author, timestamp, or message changed while the source tree stayed the same (a metadata-only move), or whether the annotation layer of an annotated tag changed. The prevalence and structure of alterations are then related to repository visibility, using GitHub star counts as a proxy for downstream exposure. The study finds that tag alteration is an active,

Mais tu résumes les chapitres !!!
Est-ce l'objectif ???

recurring practice across the corpus rather than a rare outlier confined to a few repositories, and that moves that change the source tree behind a stable version string are the primary security concern. Full prevalence figures, the taxonomy of alteration forms, and downstream consequence evidence from the Nixpkgs cross-analysis and the tj-actions case study appear in Chapter 3.

1.4.2 Commit Graphs Are Not Append-Only

If release labels can drift silently, what about the commit histories those labels anchor? The second study extends the mutability analysis from the reference boundary — tags and branch heads — inward to the commit graph itself. Distributed version control systems are, for the most part, append-only in ordinary collaborative use: developers add commits and share them through push and pull operations. Yet the same system also permits substituting new commits for old ones and overwriting the shared history with the result, through operations such as rebase, amend, interactive squash, and force-push. When applied to shared branches, these operations create a different class of integrity questions: commit identities, parent links, authorship fields, and timestamps are routinely treated as an audit trail, yet they are not append-only facts about a repository; they are whatever the current object graph says after the most recent rewrite [58]. The central questions are: How often are public Git histories observably altered? Which branches and repositories are most affected? What kinds of alterations occur? And are there recurrent patterns with security consequences?

The study analyses 111 M public Git repositories archived in Software Heritage as of 2024-08-23 [50]. For each origin, consecutive snapshot pairs are compared to detect *history alterations*: situations in which at least one commit reachable from a branch head in an earlier snapshot is absent from the same branch in a later snapshot. The directly altered commits — as opposed to their descendants, which acquire new identifiers through the snowball effect — are identified as *root-cause commits*. A mixed-method taxonomy is applied to classify each root-cause commit as a *DIR* alteration (at least one versioned file changed), a *META* alteration (only metadata such as author name, timestamp, or commit message changed), or a *Different Branch Name* alteration (the same content appears under a different branch). The study further examines recurring security-relevant patterns among DIR alterations: files whose names match naming conventions for credentials and private keys, and files whose names suggest license text. The study finds that history alteration at the commit-graph level is quantitatively non-negligible, distributed across branch types including the default branch, and more likely in more visible repositories. It also produces *GitHistorian*, a command-line tool that operationalises alteration detection on the Software Heritage graph for practitioner use. Full prevalence figures, the alteration taxonomy, and the security-relevant case studies appear in Chapter 4.

1.4.3 Removal Is Not Revocation

The second study establishes that removals of files whose names suggested secrets are detectable at scale in public repository archives, and that the altered files remain recoverable from Software Heritage regardless of what the maintainer has done to the current branch. What it cannot establish is whether the credentials in those files remain *usable*. A history rewrite is an intervention on the repository record; credential rotation is an independent intervention on the authority the leaked value grants. These are separate actions, and whether developers who perform the former also perform the latter — and in what order — is not answered by observing repository state alone. This gap motivates the third and final study.

The study works from the corpus of removals of files whose names suggested secrets, identified in the second study, and applies a multi-stage pipeline to determine what fraction of the altered files contain detectable credential material. Candidate files are selected through refined filename heuristics that combine evidence levels, from naming conventions with near-certain semantic content (`id_rsa`, `.pem`) down to broad extension classes (`.env`, `.py`). Their archived content is retrieved from Software Heritage’s content store by content identifier, deduplicated, and scanned with two independent secret detectors, *gitleaks* and *TruffleHog*. Findings are then linked back to the specific repository revisions and paths where the files that contained credentials appeared, providing provenance context for each detected secret. A staged validation protocol frames the question of whether detected credentials remain active: Tier A applies offline syntactic checks that require no network access and serve to reduce scanner false positives; Tier B contacts providers through approved security disclosure channels and asks them to verify and revoke credentials on their side; Tier C allows a read-only network check conducted by the researcher only after explicit written provider authorisation. The detection pipeline confirms that a non-negligible fraction of archived candidate blobs contain scanner-detectable credential material, and Tier A offline validation narrows the private-key population before any network probe. Whether those credentials remain active is the central open question, to be addressed through the Tier B and Tier C campaign; the completed artifacts are the detection pipeline, the Tier A results, and the responsible disclosure framework (Chapter 5).

1.5 Thesis Contributions

The three studies yield empirical findings, methodological artifacts, and practical lessons for software supply-chain security. What follows is a prospective statement of what this thesis contributes; the empirical chapters substantiate each claim in full.

1.5.1 Empirical Contributions

Empirically, the thesis contributes three first-of-kind measurements at ecosystem scale, each probing a deeper layer of repository mutability: the first large-scale study of Git tag alterations,

the first large-scale study of commit-graph history alterations, and the first blob-level detection study of credentials archived in Software Heritage after removal from public history. Each study pairs prevalence figures with a taxonomy of the observed alteration forms, as previewed in Section 1.4.

1.5.2 Methodological Contributions

Methodologically, the thesis contributes three reusable artifacts.

The first is a snapshot-pair comparison framework for studying repository mutability using the Software Heritage archive, specified in full in Section 3.3 and extended by reference in subsequent chapters.

The second is *GitHistorian*, a Rust-based command-line tool that operationalises history alteration detection on the Software Heritage graph for practitioner and CI integration use.

The third is a staged credential validation protocol (Tier A, Tier B, Tier C) that separates offline syntactic filtering from provider-assisted validation and from network checks conducted by the researcher under explicit authorisation.

1.5.3 Practical Scope

The combined findings bear on a single structural tension: downstream systems often treat mutable reference names as if they were content commitments, and forge-visible repository state as if it were a complete audit trail. Chapter 6 develops the practical implications for hosting platforms, maintainers, and downstream consumers; the empirical chapters supply the evidence on which those recommendations rest.

1.6 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 provides the technical background required for the empirical studies. It covers Git’s object model and reference semantics, the mechanisms and contexts in which Git history is rewritten, release engineering conventions and their relationship to build reproducibility, the role of software repositories in the modern supply chain, the problem of credential leakage in version control, and the archival infrastructure of Software Heritage. The chapter concludes with a glossary of key terms used throughout the thesis.

Chapter 3 presents the first empirical study: the tag-alteration analysis. The full methodological specification appears in Section 3.3; the subsequent empirical chapters extend the snapshot-pair framework by reference and detail only the aspects that differ. Chapter 4 presents the second empirical study: the analysis of commit-graph history alterations, including the taxonomy, the security-relevant case studies, and the *GitHistorian* tool. Chapter 5 presents the third study: the archived secrets detection and staged validation pipeline. Chapter 6 synthesises the

findings across all three studies, develops their cross-cutting implications, revisits limitations at the thesis level, and outlines directions for future research.